

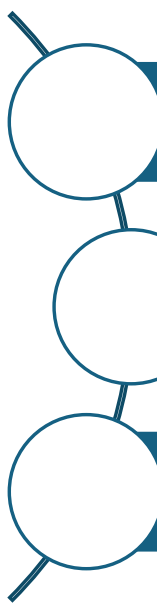


**HỌC VIỆN
NGÂN HÀNG**
BANKING ACADEMY OF VIETNAM

CHƯƠNG 2

KIỂU KIẾN TRÚC PHẦN MỀM

Hà Nội - 2026



Một số khái niệm

Phân loại Kiểu Kiến trúc phần mềm

Giới thiệu một số Kiểu kiến trúc phần mềm



- Kiểu kiến trúc phần mềm được định nghĩa là một cách tổ chức và thiết kế các thành phần của một hệ thống phần mềm.
- Kiểu kiến trúc phần mềm bao gồm:
 - Cấu trúc tổ chức của hệ thống
 - Các quy tắc và chuẩn cho việc tương tác giữa các thành phần
 - Các quyết định về cách dữ liệu được lưu trữ, xử lý và truyền tải
- Kiểu kiến trúc phần mềm không chỉ là một khung cảnh về cấu trúc, mà còn đặt ra các nguyên tắc hướng dẫn việc xây dựng, triển khai, và duy trì hệ thống phần mềm.

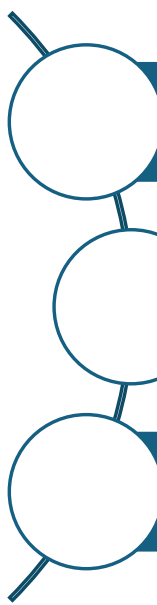


- Một số khía cạnh chính của Kiểu kiến trúc phần mềm:
 - Cấu trúc Tổ chức: Quyết định cách hệ thống được tổ chức thành các thành phần khác nhau và cách chúng tương tác với nhau. Các kiến trúc thường được biểu diễn dưới dạng biểu đồ hoặc mô hình.
 - Quyết định Kiến trúc: Bao gồm các quyết định lớn về nền tảng công nghệ, ngôn ngữ lập trình, kiểu cơ sở dữ liệu, và các quy tắc tổ chức chung.
 - Mô hình Dữ liệu: Định nghĩa cách dữ liệu được tổ chức, lưu trữ và truy cập. Bao gồm cả cấu trúc cơ sở dữ liệu và cách dữ liệu di chuyển qua hệ thống.



- Một số khía cạnh chính của Kiểu kiến trúc phần mềm:
 - Giao diện Người dùng: Đặc tả cách người dùng tương tác với hệ thống. Bao gồm cả giao diện người dùng và các thành phần liên quan.
 - Bảo trì và Mở rộng: Định nghĩa cách hệ thống có thể được bảo trì và mở rộng trong tương lai. Bao gồm cả khả năng thêm mới các tính năng và thay đổi mà không ảnh hưởng đến toàn bộ hệ thống.
 - Hiệu suất và Độ tin cậy: Bao gồm các quyết định về hiệu suất hệ thống và khả năng đảm bảo tính tin cậy và ổn định.

Kiến trúc phần mềm đóng vai trò quan trọng trong việc đảm bảo sự hiểu rõ và quản lý chặt chẽ của một dự án phần mềm, giúp tăng tính linh hoạt, tái sử dụng mã nguồn và giảm rủi ro trong quá trình phát triển và duy trì hệ thống.



Một số khái niệm

Phân loại Kiểu Kiến trúc phần mềm

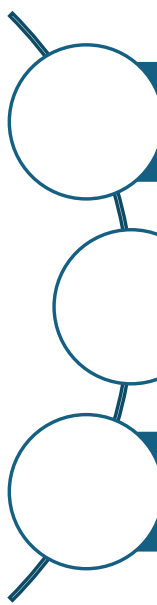
Giới thiệu một số Kiểu kiến trúc phần mềm



- Phân loại Kiểu kiến trúc phần mềm:
 - Hệ thống luồng dữ liệu (DataFlow Systems):
 - Tuần tự hàng loạt (Batch sequential); đường ống (Pipes) và bộ lọc (Filters);...
 - Hệ thống Gọi và Trả về (Call & Return Systems):
 - Chương trình chính (Main program) và Chương trình con (Subroutine); hệ thống hướng đối tượng (OO systems);...
 - Các thành phần độc lập (Independent components):
 - Quy trình giao tiếp (Communicating process); Hệ thống sự kiện (Event systems)



- Phân loại Kiểu kiến trúc phần mềm:
 - Máy ảo (Virtual machines):
 - Thông dịch viên; Hệ thống dựa trên quy tắc
 - Hệ thống tập trung vào dữ liệu (Data-centered-systems):
 - Cơ sở dữ liệu (Databases); Hệ thống siêu văn bản (Hypertext systems);...



Một số khái niệm

Phân loại Kiểu Kiến trúc phần mềm

Giới thiệu một số Kiểu kiến trúc phần mềm

Hệ thống Luồng dữ liệu (DataFolow Systems)

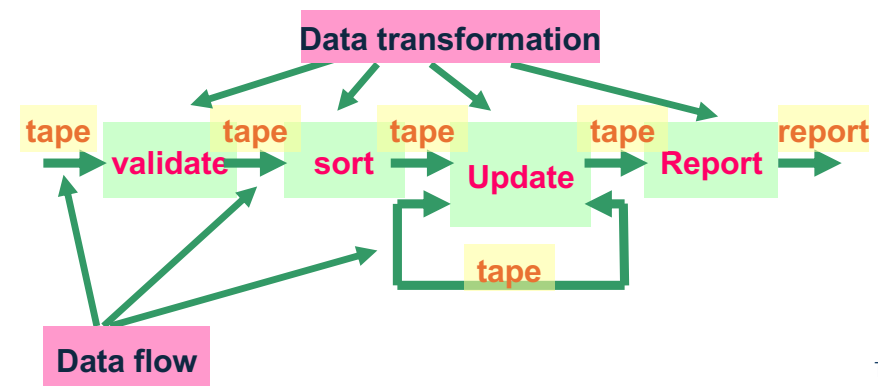


Kiểu kiến trúc tuần tự hàng loạt





- Kiểu kiến trúc tuần tự hàng loạt:
 - Đây là một kiểu kiến trúc trong lập trình và xử lý dữ liệu, trong đó các công việc hoặc nhiệm vụ được thực hiện theo từng lô (batch) một. Thay vì thực hiện công việc một cách đồng thời hoặc liên tục, kiến trúc này tập trung vào việc chia công việc thành các nhóm và thực hiện chúng tuần tự.
 - Đây là một trong những kiểu lâu đời nhất trong thiết kế hệ thống máy tính. Nó xuất hiện trong những ngày đầu của quá trình xử lý dữ liệu khi những hạn chế của thiết bị máy tính đòi hỏi các vấn đề lớn phải được chia thành các thành phần có thể tách rời và có thể giao tiếp bằng cách truyền băng từ.

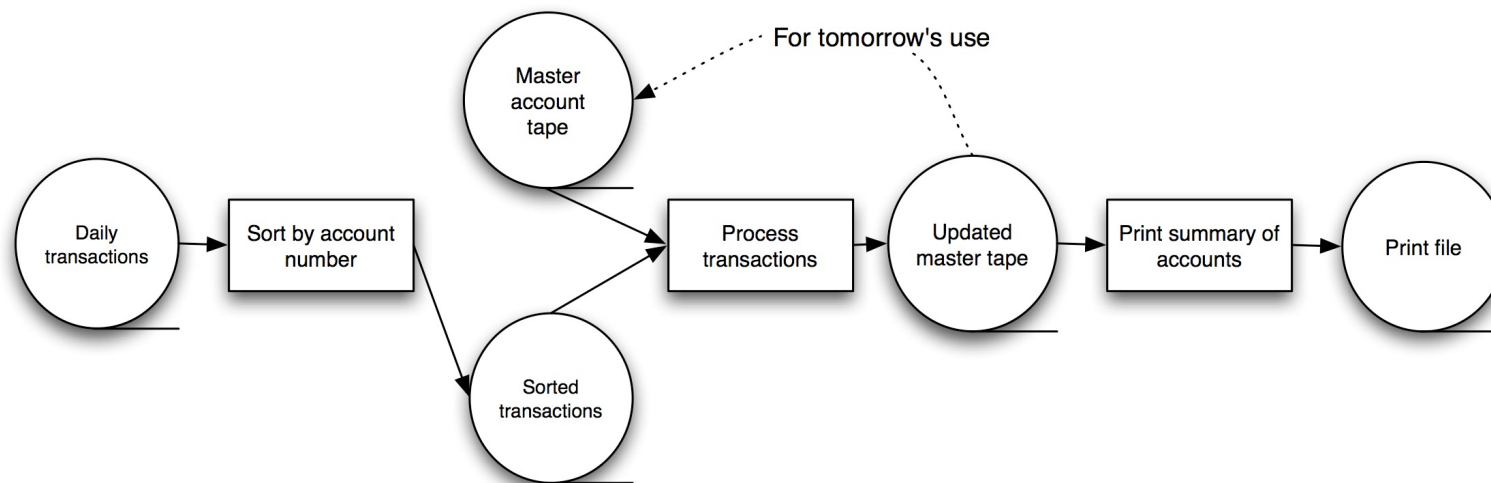




- Đặc điểm Kiểu kiến trúc tuần tự hàng loạt:
 - Các bước xử lý là các chương trình độc lập.
 - Mỗi bước sẽ hoàn thành trước khi bắt đầu bước tiếp theo.
 - Toàn bộ dữ liệu được truyền giữa các bước.
 - Vì các tính năng như trên nên không cần phải đồng bộ hóa các thành phần của nó.
 - Vì một số thành phần chỉ có thể chạy theo trình tự chứ không thể chạy đồng thời nên hiệu suất có thể kém hơn so với các chương trình chạy đồng thời trên nhiều thành phần.
 - Phần mềm được thiết kế theo cấu trúc lô (Batch) tuần tự (Sequential) không phù hợp với các hệ thống xử lý dữ liệu theo thời gian thực.



- Tuần tự hàng loạt:
 - Ứng dụng điển hình: Kiến trúc này thường được sử dụng trong các ứng dụng xử lý hàng loạt dữ liệu, quy trình sản xuất, hoặc các hệ thống nơi quá trình phải được kiểm soát và đồng bộ.





```
import time
```

Định nghĩa lớp công việc

```
class Job:
    def __init__(self, name):
        self.name = name

    def execute(self):
        print(f"Executing job: {self.name}")
        # Giả sử công việc mất thời gian thực hiện
        time.sleep(1)
```

Lớp quản lý Batch Sequential

```
class BatchProcessor:
    def __init__(self):
        self.job_list = []

    def add_job(self, job):
        self.job_list.append(job)

    def process_batch(self):
        print("Starting Batch Processing...")

        start_time = time.time()

        for job in self.job_list:
            job.execute()

        end_time = time.time()

        print(f"Batch Processing completed in {end_time - start_time:.2f} seconds.")
```

Tạo các công việc

```
job1 = Job("Job 1")
job2 = Job("Job 2")
job3 = Job("Job 3")
```

Tạo và cấu hình Batch Processor

```
batch_processor = BatchProcessor()
batch_processor.add_job(job1)
batch_processor.add_job(job2)
batch_processor.add_job(job3)
```

Thực hiện Batch Sequential và đo thời gian

```
batch_processor.process_batch()
```

Kết quả thực thi

```
Starting Batch Processing...
Executing job: Job 1
Executing job: Job 2
Executing job: Job 3
Batch Processing completed in 3.02 seconds.
```



- Tóm tắt (Summary): Các chương trình riêng biệt được thực hiện theo thứ tự; dữ liệu được truyền dưới dạng tổng hợp từ chương trình này sang chương trình tiếp theo.
- Thành phần (Components): Các chương trình độc lập.
- Kết nối (Connectors): Con người kết nối thủ công giữa các chương trình
- Các phần tử dữ liệu (Data elements): Các phần tử tổng hợp, rõ ràng được truyền từ thành phần này sang thành phần tiếp theo sau khi hoàn thành quá trình thực thi chương trình sản xuất.
- Cấu trúc liên kết (Topology): Tuyến tính.
- Các ràng buộc bổ sung (Additional constraints): Một chương trình chạy tại một thời điểm cho đến khi hoàn thành.
- Chất lượng thu được (Qualities yielded): Tăng tính tuân thủ & sự đơn giản khi thực thi.
- Kiểu ứng dụng điển hình (Typical uses): Xử lý giao dịch trong hệ thống tài chính.
- Cảnh báo (Cautions): Khi cần có sự tương tác giữa các thành phần; khi có thể hoặc cần có xử lý đồng thời giữa các thành phần.
- Mối quan hệ với ngôn ngữ lập trình hoặc môi trường (Relations to programming languages or environments): Không có.

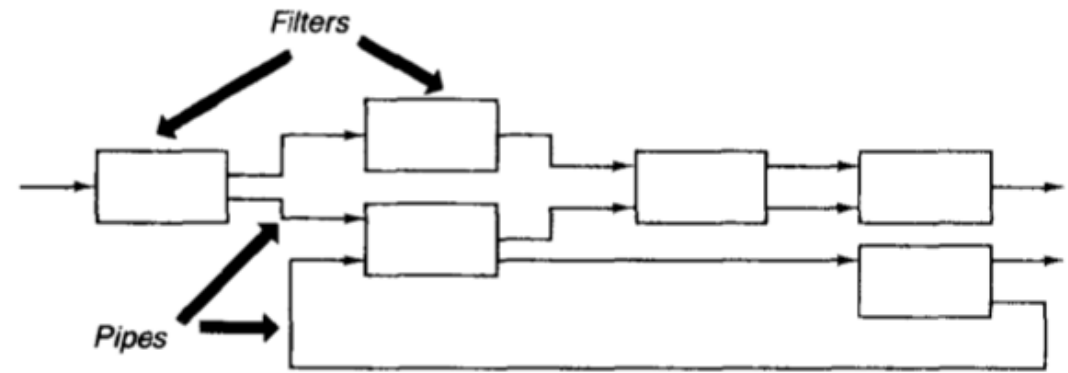


Kiểu kiến trúc Đường ống (Pipes) và Bộ lọc (Filters)



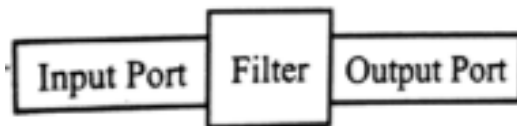


- Kiểu kiến trúc Đường ống và Bộ lọc (Pipe-and-Filter) là một kiểu kiến trúc phần mềm trong đó:
 - Hệ thống được chia thành các thành phần nhỏ gọi là "filters" (bộ lọc), mỗi filter thực hiện một nhiệm vụ cụ thể và dữ liệu được chuyển đổi qua các ống (pipes) giữa các filters.
 - Mỗi filter độc lập và không biết đến sự tồn tại của các filter khác, điều này tạo ra sự tách biệt và linh hoạt trong thiết kế hệ thống.

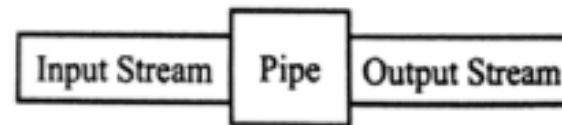




- Cấu trúc bộ lọc như trong Hình 1. Bộ lọc bao gồm cổng đầu vào, bộ lọc và cổng đầu ra. Phần cổng đầu vào có nhiệm vụ lưu trữ dữ liệu đang chờ xử lý, bộ lọc có nhiệm vụ xử lý dữ liệu, cổng đầu ra có nhiệm vụ lưu trữ dữ liệu đã được xử lý.
- Trách nhiệm của đường ống là vận chuyển dữ liệu, cấu trúc của nó được hiển thị trong Hình 2. Đường ống bao gồm Luồng đầu vào và Luồng đầu ra, và đường ống thành phần ống chịu trách nhiệm truyền dữ liệu từ luồng đầu vào sang luồng đầu ra



Hình 1



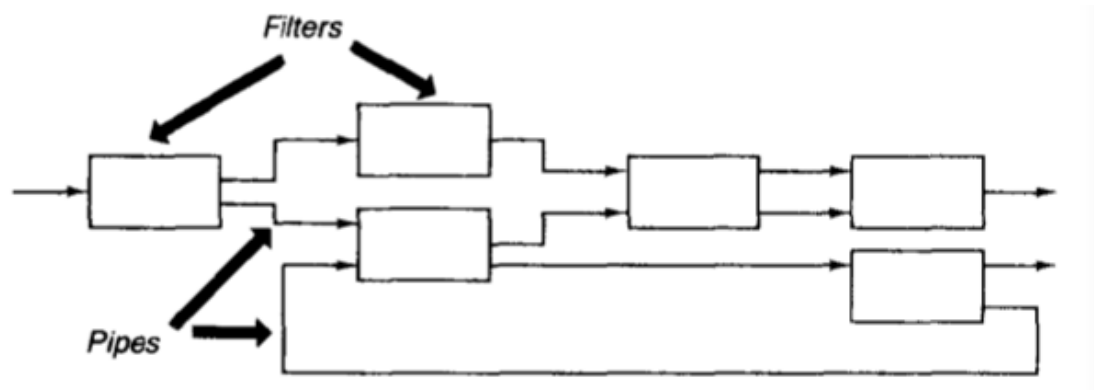
Hình 2



- Các thành phần chính Kiểu Kiến trúc Pipe-and-Filter:
 - Filter (Bộ lọc):
 - Filters là các thành phần độc lập thực hiện một nhiệm vụ cụ thể.
 - Mỗi filter chỉ biết đến dữ liệu được đưa vào và được trả ra từ pipe của mình.
 - Filters có thể được tái sử dụng và thay đổi mà không ảnh hưởng đến các filter khác trong hệ thống.
 - Pipe (Ống):
 - Pipes là cách dữ liệu được chuyển giữa các filter.
 - Dữ liệu được truyền qua pipes một cách tuần tự, mỗi pipe kết nối một đầu ra của một filter với đầu vào của một filter khác.
 - Pipes chịu trách nhiệm về việc truyền dữ liệu giữa các filter mà không cần sự can thiệp của chúng.



- Các thành phần chính Kiểu Kiến trúc Pipe-and-Filter:
 - Source Filter (Filter nguồn):
 - Là filter đầu tiên trong chuỗi, chịu trách nhiệm về việc sinh dữ liệu hoặc đầu vào cho hệ thống.
 - Sink Filter (Filter đích):
 - Là filter cuối cùng trong chuỗi, xử lý dữ liệu đầu ra cuối cùng của hệ thống.





- Ưu điểm
 - Tính tái sử dụng: Filters có thể được sử dụng lại trong nhiều hệ thống khác nhau vì chúng độc lập và không phụ thuộc vào các thành phần khác.
 - Dễ mở rộng: Thêm hoặc thay đổi filters mà không ảnh hưởng đến các phần khác của hệ thống.
 - Dễ duy trì và kiểm soát: Mỗi filter tập trung vào một nhiệm vụ cụ thể, giảm sự phức tạp của mã nguồn và tăng khả năng kiểm soát.



- Nhược điểm:
 - Độ trễ có thể tăng khi dữ liệu phải đi qua nhiều filters
 - Hoặc sự phức tạp có thể tăng nếu không quản lý cẩn thận.



```
import time

# Định nghĩa các filters
def filter1(data):
    result = data * 2
    print(f"Filter 1: {data} -> {result}")
    return result

def filter2(data):
    result = data + 5
    print(f"Filter 2: {data} -> {result}")
    return result

def filter3(data):
    result = data ** 2
    print(f"Filter 3: {data} -> {result}")
    return result

# Định nghĩa lớp quản lý Pipe-and-Filter
class PipeAndFilterManager:
    def __init__(self):
        self.filters = []

    def add_filter(self, filter_func):
        self.filters.append(filter_func)
```

```
def process_data(self, data):
    print("Starting Pipe-and-Filter Processing...")

    start_time = time.time()

    for filter_func in self.filters:
        data = filter_func(data)

    end_time = time.time()

    print(f"Pipe-and-Filter Processing completed in {end_time -
start_time:.2f} seconds.")

# Tạo đối tượng quản lý Pipe-and-Filter
pipe_and_filter_manager = PipeAndFilterManager()

# Thêm các filters vào quản lý
pipe_and_filter_manager.add_filter(filter1)
pipe_and_filter_manager.add_filter(filter2)
pipe_and_filter_manager.add_filter(filter3)

# Dữ liệu đầu vào
input_data = 3

# Thực hiện Pipe-and-Filter và đo thời gian
pipe_and_filter_manager.process_data(input_data)
```



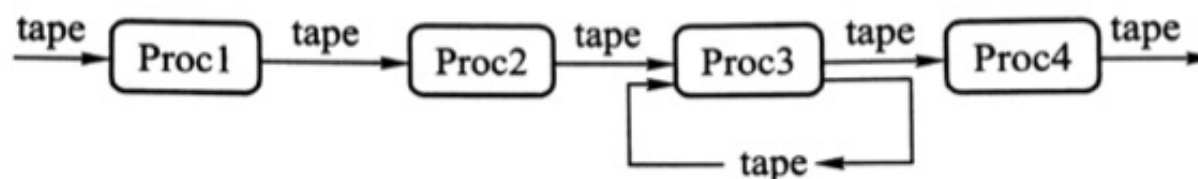

Kết quả

```
Starting Pipe-and-Filter Processing...  
Filter 1: 3 -> 6  
Filter 2: 6 -> 11  
Filter 3: 11 -> 121  
Pipe-and-Filter Processing completed in 0.00 seconds.
```

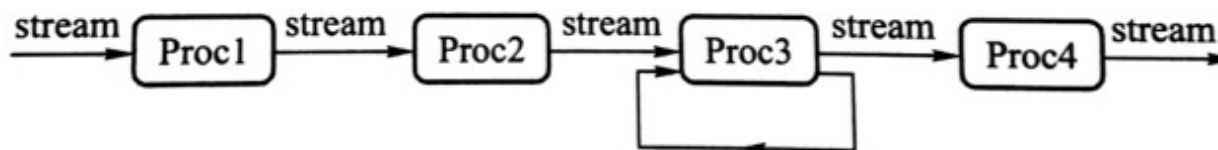


- Tóm tắt (Summary): Các chương trình riêng biệt được thực hiện đồng thời; dữ liệu được truyền dưới dạng luồng từ chương trình này sang chương trình tiếp theo.
- Thành phần (Components): Các chương trình độc lập, được gọi là bộ lọc.
- Kết nối (Connectors): Bộ định tuyến rõ ràng của luồng dữ liệu; dịch vụ do hệ điều hành cung cấp.
- Yếu tố dữ liệu (Data): Không rõ ràng; phải là luồng dữ liệu (tuyến tính). Trong triển khai Unix/Linux/DOS điển hình, các luồng phải là văn bản.
- Cấu trúc liên kết (Topology): Đường ống, mặc dù có thể có LK kiểu chữ T.
- Chất lượng thu được (Qualities yielded): Các bộ lọc độc lập lẫn nhau. Cấu trúc đơn giản của các luồng dữ liệu đến và đi tạo điều kiện thuận lợi cho việc kết hợp các bộ lọc mới cho các ứng dụng tổng hợp mới.
- Kiểu ứng dụng điển hình (Typical uses): Phổ biến trong lập trình ứng dụng hệ điều hành.
- Cảnh báo (Cautions): Khi phải trao đổi cấu trúc dữ liệu phức tạp giữa các bộ lọc; khi cần có sự tương tác giữa các chương trình.
- Mối quan hệ với ngôn ngữ lập trình hoặc môi trường (Relations to programming languages or environments): Phổ biến trong hệ điều hành Unix.

So sánh Kiểu tuần tự hàng loạt với “Đường ống (Pipes) và Bộ lọc (Filters)”



Hình 1: Cấu trúc Kiến trúc hệ thống tuần tự hàng loạt



Hình 2: Cấu trúc Kiến trúc hệ thống Đường ống và bộ lọc

So sánh Kiểu tuần tự hàng loạt với “Đường ống (Pipes) và Bộ lọc (Filters)”



- Điểm giống nhau giữa hai cấu trúc nằm ở tính độc lập lẫn nhau của các module xử lý. Trong các hệ thống lô và kiến trúc đường ống và bộ lọc, các quy trình độc lập với nhau, nghĩa là, trong kiến trúc hệ thống lô, mỗi quy trình không gọi các quy trình khác; trong kiến trúc đường ống và bộ lọc, mỗi bộ lọc không gọi các bộ lọc khác.

So sánh Kiểu tuần tự hàng loạt với “Đường ống (Pipes) và Bộ lọc (Filters)”



- Sự khác biệt giữa hai kiến trúc là:
 - Xử lý dữ liệu là khác nhau.
 - Trong hệ thống xử lý hàng loạt tuần tự, chỉ có bước cuối cùng được thực hiện hoàn chỉnh, bước tiếp theo mới có thể bắt đầu và dữ liệu được truyền dưới dạng khối. Thông thường có bộ lưu trữ trung gian cho mỗi bước trung gian. Mỗi quy trình không hoạt động cùng lúc và chỉ khi quá trình xử lý liên kế trước đó hoàn tất thì quy trình mới có thể bắt đầu hoạt động.
 - Trong kiến trúc ống và bộ lọc, các bộ lọc chuyển đổi dữ liệu dần dần theo từng luồng, trong đó mỗi bộ lọc hoạt động đồng thời.

So sánh Kiểu tuần tự hàng loạt với “Đường ống (Pipes) và Bộ lọc (Filters)”



- Sự khác biệt giữa hai kiến trúc là:
 - Lượng dữ liệu được xử lý là khác nhau.
 - Lượng dữ liệu được xử lý bởi hệ thống tuần tự hàng loạt bị hạn chế.
 - Lượng dữ liệu được xử lý bởi kiến trúc đường ống và bộ lọc là vô hạn.

